Universidad de Alicante
Escuela Politécnica Superior

TECNOLOGÍA Y ARQUITECTURA ROBÓTICA (TAR)

Práctica 2

Turtlebot 3 en Entorno

Autor: Hernández Delgado, Jaime
Álvarez Bernabeu, Daniel
Mazón Muñoz, Manuel Jesús

Curso: 2025–2026

Repositorio GitHub:
https://github.com/nickhernd/TAR
Video de resolución de laberintos:
https://youtu.be/P-GyLycobWg
Video de aparcamiento y figuras corregidas:
https://youtu.be/5ZaJrfdw7kY

5 de abril de 2026

Índice general

1. Parte 1: Primeros Pasos con el Turtlebot y Odometría **2**
1.1. Preguntas teóricas previas 2
1.2. Ejercicio 1: Figuras Geométricas 3
1.3. Ejercicio 2: Monitorización y Gráficas 7
1.4. Ejercicio 3: Aparcamiento Autónomo 7
1.5. Ejercicio 4: Corrección de Errores con Controlador PID 10

2. Parte 2: Resolución de un Laberinto **14**
2.1. Preguntas teóricas previas 14
2.2. Ejercicio 1: Resolución del Laberinto 1 — Algoritmo Wall Follower . . . 15
2.3. Ejercicio 2: Laberinto 2 y análisis de limitaciones del Wall Follower . . 17
2.4. Ejercicio 3: Algoritmo de Pledge para el Laberinto 2 18
2.5. Ejercicio 4: Comparativa Waffle vs. Burger en el Laberinto 1 20
2.6. Ejercicio 5: Laberinto propio (maze_3) 21

1. Parte 1: Primeros Pasos con el Turtlebot y Odometría

En esta sección se detalla la configuración inicial del entorno de simulación, la resolución de cuestiones teóricas sobre la cinemática del Turtlebot 3 y la implementación de nodos de control para trayectorias básicas y monitorización de odometría.

1.1 Preguntas teóricas previas

Preguntas 1 y 2: Topic y tipo de mensaje de movimiento

El topic en el cual se debe publicar la información para que el Turtlebot 3 se mueva es `/cmd_vel`. Podemos verificar tanto el topic como el tipo de mensaje ejecutando los siguientes comandos en una terminal paralela mientras el nodo de teleoperación (`teleop_keyboard`) está activo:

```
ros2 topic list
ros2 topic info /cmd_vel
```

El tipo de mensaje que se publica en este topic es `geometry_msgs/msg/Twist`. Este estándar de ROS contiene dos vectores tridimensionales: uno para expresar la velocidad lineal (`linear`) en los ejes X, Y, Z; y otro para la velocidad angular (`angular`) alrededor de dichos ejes.

Pregunta 3: Tipos de movimientos y ejes positivos

El Turtlebot 3, al poseer una configuración de tracción diferencial, está restringido a realizar movimientos en un plano 2D. Por tanto, emplea principalmente:

- **Movimiento lineal:** Desplazamiento hacia adelante o hacia atrás a lo largo de su eje longitudinal (`linear.x`).
- **Movimiento angular:** Rotación sobre su propio eje central (`angular.z`).

En ROS 2 se utiliza la regla de la mano derecha (estándar REP 103) para los sistemas de coordenadas. Esto implica que:

- El eje **X positivo** apunta hacia el frente del robot.
- El eje **Z positivo** apunta hacia arriba. Por tanto, una velocidad angular positiva produce un giro en sentido antihorario (**hacia la izquierda**).

En consecuencia, si le enviamos al robot un comando para que gire **-20º** (una velocidad angular negativa), el robot girará en sentido horario, es decir, **hacia la derecha**.

Pregunta 4: Giro sobre su eje y cinemática

Sí, el robot gira exactamente sobre su propio eje central vertical.

Esto se debe a su configuración cinemática de **tracción diferencial** (*differential drive*). Cuando enviamos un comando puramente rotacional a través de `/cmd_vel` (con velocidad lineal a 0), el controlador base del Turtlebot traduce este mensaje y manda a los motores de las ruedas comandos de velocidad de la **misma magnitud pero de signos opuestos**.

Por ejemplo, para girar a la izquierda, la rueda derecha recibe una velocidad positiva (avanza) y la rueda izquierda recibe una velocidad negativa (retrocede), provocando que el centro instantáneo de rotación (ICR) se sitúe exactamente en el centro del eje que une ambas ruedas.

Pregunta 5: Unidades de magnitud

De acuerdo con el estándar internacional adoptado por ROS (REP 103), las magnitudes enviadas en los mensajes de tipo `Twist` utilizan el Sistema Internacional de Unidades (SI):

- **Velocidad lineal:** Se mide en metros por segundo (**m/s**).
- **Velocidad angular:** Se mide en radianes por segundo (**rad/s**).

1.2 Ejercicio 1: Figuras Geométricas

Implementación del nodo `movimiento.py`

Para cumplir con el requerimiento de un único fichero ejecutable, se ha desarrollado una clase en Python orientada a objetos (`ControladorFiguras`) que lee el argumento pasado por la línea de comandos para seleccionar la figura a trazar. El control del movimiento se realiza en *lazo abierto*, calculando el tiempo de ejecución teórico necesario para cada desplazamiento y giro.

Fundamento Matemático: Al publicar mensajes del tipo `Twist` con velocidades constantes, el tiempo de activación (t) se calcula en función de la distancia (d), el ángulo a girar (θ) y las velocidades correspondientes (v lineal y ω angular):

$$t_{lineal} = \frac{d}{v} \quad ; \quad t_{giro} = \frac{\theta}{\omega} \tag{1.1}$$

Se establecieron los siguientes parámetros para las figuras:

- **0. Línea Recta:** El robot avanza $2\,m$ a $0{,}2\,m/s$ ($t = 10\,s$).

Pregunta 6: ¿Cómo puedo comprobar que el robot haya avanzado esta distancia?

Para comprobar de forma empírica la distancia real que ha recorrido el robot en la simulación, debemos abandonar el control ciego y monitorizar sus sensores de posición espacial (odometría). Tal como sugiere la pista, al ejecutar el comando `ros2 topic list` en la terminal, identificamos el topic `/odom`.

Este topic publica mensajes del tipo `nav_msgs/msg/Odometry`. Podemos verificar el avance ejecutando el comando `ros2 topic echo /odom`. Esto imprime en pantalla, en tiempo real, la información del topic. Observando los valores de `pose.pose.position.x` y `pose.pose.position.y`, podemos comprobar las coordenadas finales del robot una vez se detiene y restarlas de su posición inicial para obtener la distancia real recorrida.

- **1. Triángulo equilátero:** El robot avanza $3\,m$ a $0{,}2\,m/s$ ($t = 15\,s$). Para cerrar la figura, el robot rota el ángulo exterior del polígono, equivalente a 120° ($2\pi/3\,rad$) a una velocidad de $0{,}5\,rad/s$. Este patrón se repite 3 veces.
- **2. Cuadrado:** El robot avanza $1\,m$ a $0{,}2\,m/s$ ($t = 5\,s$). Posteriormente, realiza un giro de 90° ($\pi/2\,rad$) a una velocidad de $0{,}5\,rad/s$. Este patrón se repite 4 veces.
- **3. Infinito:** El robot inicia avanzando $0{,}5\,m$ en línea recta ($t = 2{,}5\,s$). Aunque el enunciado menciona un giro de 60°, cinemáticamente este corresponde al ángulo interior θ. Para ejecutarlo, el robot gira sobre su eje el ángulo suplementario de 120° ($2\pi/3\,rad$). La secuencia alterna avances rectilíneos para los lados verticales ($0{,}5\,m$) y las diagonales ($1{,}0\,m$) con giros de $\pm 120°$ hasta regresar al origen con su orientación inicial.

Para garantizar la estabilidad en la simulación, se implementaron paradas de seguridad publicando velocidades nulas ($0{,}0\,m/s$, $0{,}0\,rad/s$) durante 0,5 segundos en las transiciones de movimiento lineal y angular.

Resultados en Simulación

A continuación, se presentan las capturas obtenidas en el simulador Gazebo que ilustran la trayectoria ejecutada por el robot para cada una de las figuras implementadas.

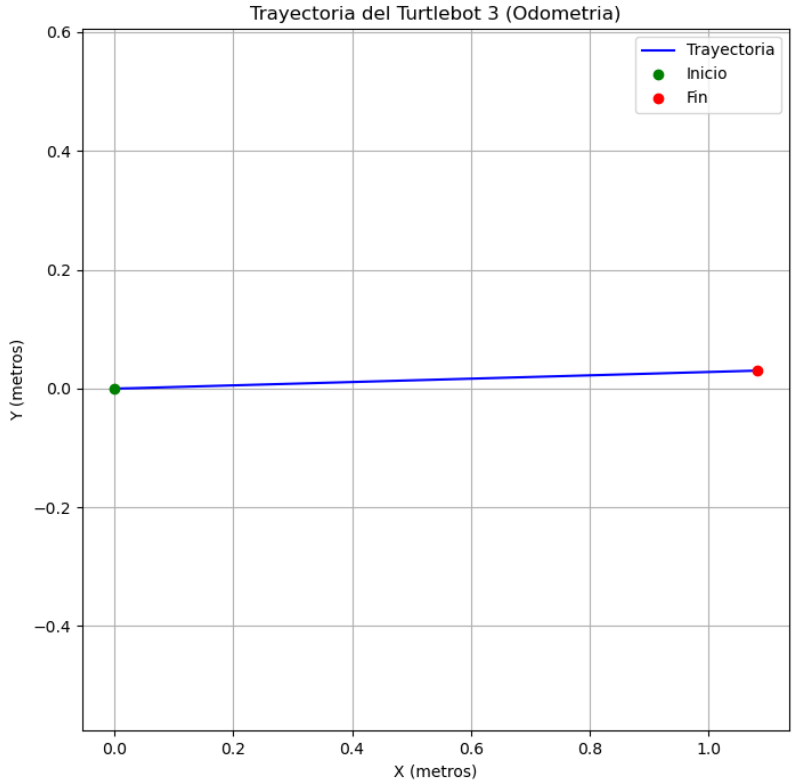

Figura 1.1: Ejecución del movimiento en línea recta (Opción 0).

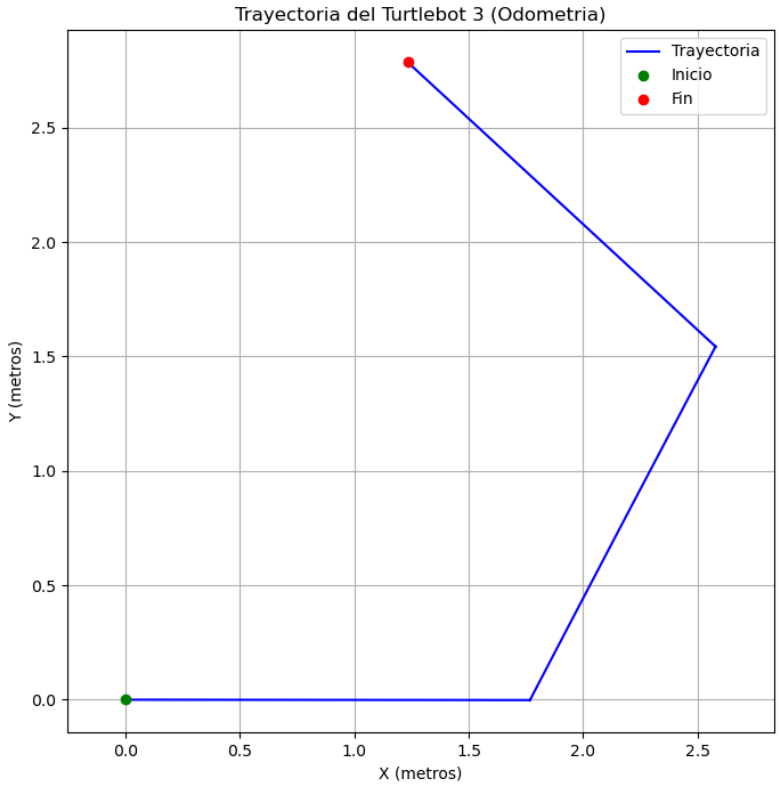

Figura 1.2: Ejecución de la trayectoria triangular (Opción 1).

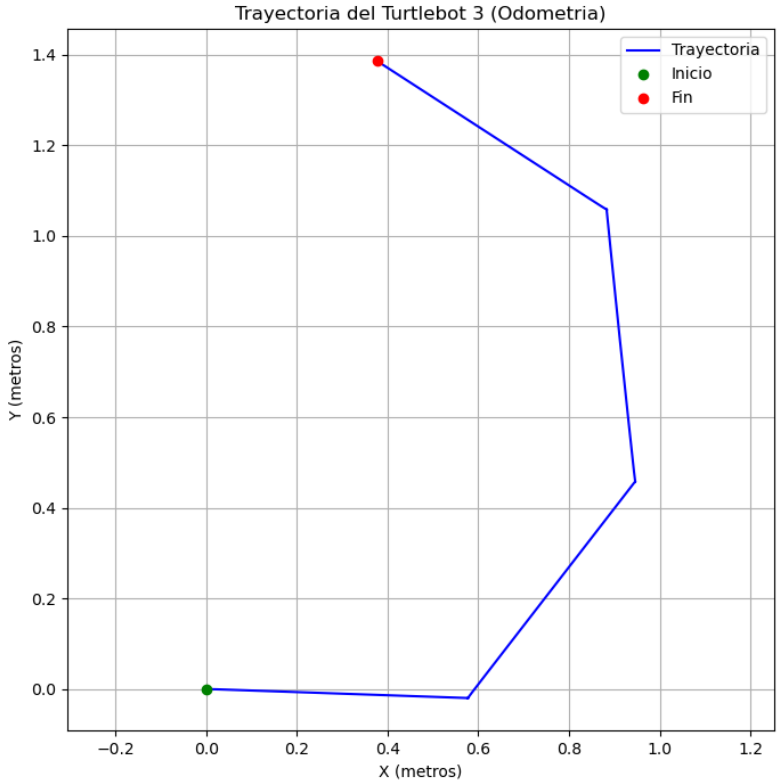

Figura 1.3: Ejecución de la trayectoria cuadrada (Opción 2).

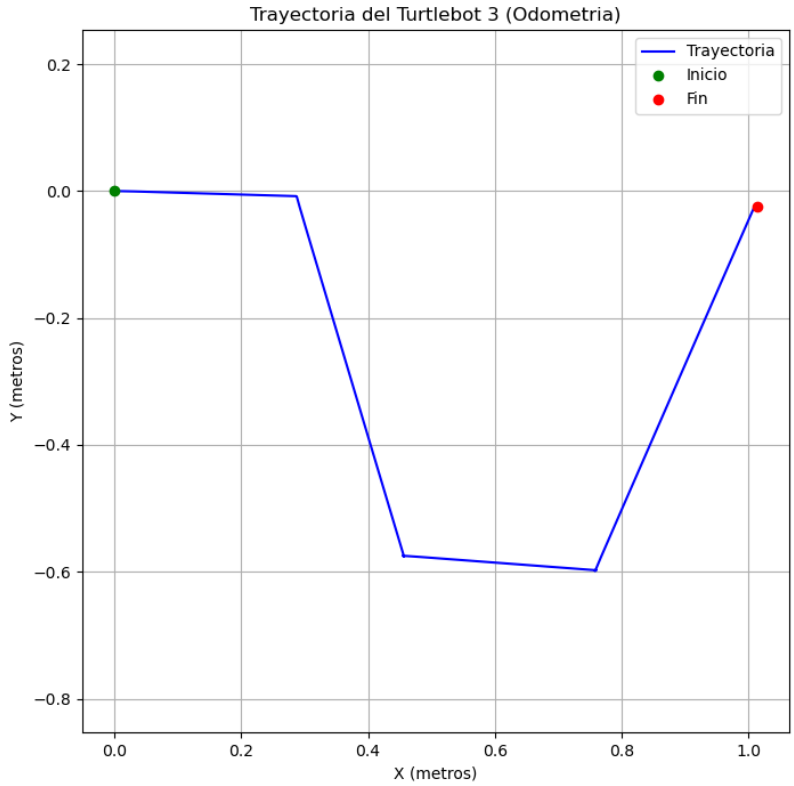

Figura 1.4: Ejecución de la trayectoria en forma de lazo o infinito (Opción 3).

1.3 Ejercicio 2: Monitorización y Gráficas

Análisis de Odometría con `dibuja_mov.py`

Se ha desarrollado un nodo que se suscribe al topic `/odom` para registrar la evolución temporal de la posición (x, y) del robot y graficar la trayectoria real mediante la librería Matplotlib.

Figura 1.5: Trayectoria real del cuadrado registrada mediante odometría. El punto verde indica el inicio y el rojo el final.

Pregunta 7: Análisis del error en la posición final

Tras ejecutar 10 veces los movimientos del cuadrado, del triángulo y del infinito monitorizando los resultados, se observa un patrón claro en las gráficas obtenidas.

¿El robot acaba en la posición original?
No. En todas las ejecuciones, el robot finaliza su trayectoria con un error de posición y orientación significativo respecto al punto de partida (marcado en las gráficas por la distancia entre el punto verde inicial y el rojo final). Además, la longitud de los trazos rectos y los ángulos de giro no alcanzan los valores teóricos programados.

¿A qué puede deberse esto?
Este error sistemático se debe a la naturaleza del **control en lazo abierto (open-loop control)** y a las físicas del simulador, concretamente a los siguientes factores:

1. **Factor de Tiempo Real (RTF) en Gazebo:** El código implementado se basa en el reloj del sistema operativo (`time.time()`). Sin embargo, Gazebo puede ejecutar la física de la simulación a un ritmo más lento (RTF $<$1.0) dependiendo de la carga computacional. Esto provoca que los comandos de velocidad se interrumpan antes de que el robot complete el movimiento en el tiempo simulado.

2. **Inercia y dinámica del robot:** Matemáticamente se asume un paso instantáneo de 0,0 a 0,2 m/s. Físicamente, el robot necesita una rampa de aceleración. Durante esta fase transitoria, la velocidad real es inferior a la comandada, resultando en una distancia recorrida menor a la calculada teóricamente.

3. **Falta de retroalimentación (Feedback):** Al ser un sistema ciego, el controlador no verifica su posición actual ni corrige las desviaciones causadas por deslizamientos de las ruedas o errores de fricción en la simulación.

1.4 Ejercicio 3: Aparcamiento Autónomo

Se ha desarrollado el nodo `aparcamiento.py` dentro del paquete `p3_pkg`. El objetivo es que el Turtlebot 3 ejecute una maniobra de aparcamiento perpendicular de forma autónoma, entrando en una plaza delimitada por paredes laterales y una pared trasera.

Configuración del entorno

Para cargar el entorno de simulación con la plaza de aparcamiento, se han copiado los directorios `models`, `worlds` y `launch` proporcionados en la carpeta `Parte_1` al directorio raíz del paquete `p3_pkg`. Además, se ha modificado el archivo `setup.py` para incluir estos recursos como `data_files` y se ha añadido la dependencia `ros2launch` en el `package.xml`. El entorno se lanza con:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch p3_pkg parking_tb3.launch.py
```

Arquitectura del nodo

El nodo `aparcamiento.py` implementa una **máquina de estados** controlada por un timer a 10 Hz. Utiliza dos fuentes de información sensorial:

- **Odometría** (`/odom`): proporciona la posición (x, y) y la orientación (yaw) del robot, obtenida mediante la conversión del cuaternión a ángulo de Euler. Se utiliza para calcular distancias recorridas y ángulos girados.
- **LiDAR** (`/scan`): proporciona un array de 360 distancias. Se procesan sectores específicos del escaneo: los 20 rayos centrales (frente) y los 20 rayos alrededor de 270° (derecha) para obtener lecturas robustas, filtrando valores inválidos y tomando el mínimo.

Máquina de estados

La maniobra de aparcamiento se descompone en cinco estados secuenciales:

1. **WAITING**: El nodo espera a recibir al menos un escaneo del LiDAR antes de iniciar la maniobra. Esto garantiza que el robot no se mueva antes de disponer de datos sensoriales válidos.
2. **ADVANCE_TO_SPOT**: El robot avanza en línea recta a $0{,}15\,m/s$. La distancia recorrida se calcula mediante la distancia euclídea entre la posición actual y la posición al inicio de esta fase:

$$d = \sqrt{(x_{actual} - x_{inicio})^2 + (y_{actual} - y_{inicio})^2} \tag{1.2}$$

   Cuando $d \geq 1{,}5\,m$, el robot se detiene. Esta distancia ha sido calibrada experimentalmente para que el robot quede alineado con el centro de la entrada de la plaza, dado que la posición de spawn es fija en la simulación.
3. **TURN_RIGHT**: El robot gira en sentido horario (velocidad angular negativa, $-0{,}3\,rad/s$) hasta completar un giro de 90 ($\pi/2\,rad$). El ángulo girado se calcula comparando el yaw actual con el yaw guardado al inicio del giro, normalizando la diferencia al rango $[-\pi, \pi]$ para evitar discontinuidades. El giro se considera completado cuando el error es inferior a $0{,}05\,rad$ (≈ 3).

4. **ENTER_SPOT**: El robot avanza hacia el interior de la plaza a velocidad reducida (0,105 m/s, un 70 % de la velocidad nominal) para mayor precisión. En cada iteración, el LiDAR mide la distancia frontal. Cuando esta distancia es inferior a 0,35 m (distancia de seguridad a la pared trasera), el robot se detiene.
5. **PARKED**: Estado final. El robot publica velocidad cero de forma indefinida.

Fragmento de código relevante

El bucle principal de control (`control_loop`) evalúa el estado actual en cada tick del timer y publica el comando de velocidad correspondiente:

```
def control_loop(self):
if self.scan_data is None:
return
twist = Twist()

if self.state == 'ADVANCE_TO_SPOT':
if self.get_distance_traveled() < 1.5:
twist.linear.x = self.linear_speed
else:
self.stop_robot()
self.state = 'TURN_RIGHT'
self.start_yaw = self.yaw

elif self.state == 'TURN_RIGHT':
if self.get_angle_turned() < math.pi/2 - 0.05:
twist.angular.z = -self.angular_speed
else:
self.stop_robot()
self.state = 'ENTER_SPOT'

elif self.state == 'ENTER_SPOT':
if self.get_front_distance() > self.stop_distance:
twist.linear.x = self.linear_speed * 0.7
else:
self.stop_robot()
self.state = 'PARKED'

self.cmd_pub.publish(twist)
```

Resultados

La maniobra se ejecuta correctamente tanto con el modelo Waffle como con el modelo Burger. El robot se detiene dentro de la plaza sin colisionar con las paredes laterales ni con la pared trasera. La distancia de avance de 1,5 m fue calibrada iterativamente hasta lograr que el robot quedase centrado en la entrada de la plaza.

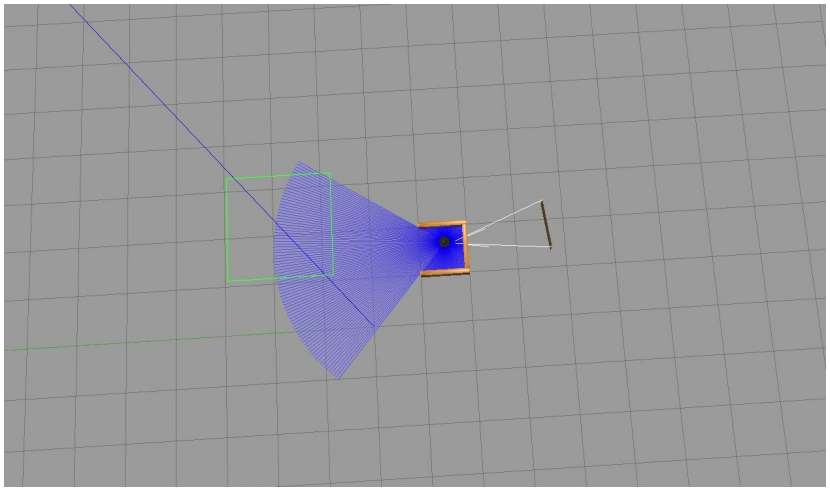

Figura 1.6: Turtlebot 3 Waffle aparcado correctamente dentro de la plaza.

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion con el entorno de parking
export TURTLEBOT3_MODEL=waffle
ros2 launch p3_pkg parking_tb3.launch.py

# Terminal 2 - Nodo de aparcamiento
ros2 run p3_pkg aparcamiento
```

1.5 Ejercicio 4: Corrección de Errores con Controlador PID

Como se ha evidenciado en los Ejercicios 1 y 2, el control en lazo abierto acumula errores de forma sistemática. En este ejercicio se implementa un **controlador PID** que corrige las trayectorias en tiempo real utilizando la retroalimentación de la odometría.

Problema del lazo abierto

En el Ejercicio 1, el robot calcula el tiempo necesario para recorrer una distancia o girar un ángulo a velocidad constante ($t = d/v$ o $t = \theta/\omega$). Sin embargo, la velocidad real del robot no coincide con la comandada debido a la inercia, la fricción y el *Real Time Factor* de Gazebo. Tras 10 repeticiones (Ejercicio 2), el error acumulado hace que el robot termine lejos del origen.

Solución: control en lazo cerrado

El nodo `movimiento_controlado.py` sustituye el control temporal por un **control basado en el error**, donde el robot mide continuamente la diferencia entre su estado actual y el objetivo, y ajusta sus velocidades proporcionalmente.

Controlador PID para giros

Para los giros se implementa un controlador PID completo. El error se define como la diferencia angular normalizada entre la orientación objetivo y la actual:

$$e(t) = \text{normalize}\left(\theta_{objetivo} - \theta_{actual}\right) \tag{1.3}$$

La señal de control (velocidad angular) se calcula como:

$$\omega(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau)\, d\tau + K_d \cdot \frac{de(t)}{dt} \tag{1.4}$$

Cada componente cumple una función específica:

- **Proporcional** ($K_p = 2{,}0$): produce una corrección proporcional al error actual. Si al robot le faltan 45 para alcanzar el objetivo, gira rápido; si le falta 1, gira muy despacio. Esto evita que el robot se pase del ángulo objetivo.
- **Integral** ($K_i = 0{,}01$): acumula el error a lo largo del tiempo. Si la componente proporcional no es suficiente para eliminar un error residual pequeño (por ejemplo, 0,5 que persiste porque la velocidad proporcional resultante no vence la fricción), la integral va creciendo hasta compensarlo. Se limita a $\pm 1{,}0$ (*anti-windup*) para evitar que se acumule excesivamente.
- **Derivativa** ($K_d = 0{,}1$): evalúa la tasa de cambio del error. Si el error está disminuyendo rápidamente (el robot se acerca al objetivo a gran velocidad), la derivada actúa como un freno, amortiguando la respuesta para evitar sobreoscilación.

```
# Giro con PID
error = self.normalize_angle(self.target_yaw - self.yaw)

self.integral_error += error * dt
self.integral_error = max(-1.0, min(1.0, self.integral_error))
derivative = (error - self.prev_error) / dt
self.prev_error = error

angular_vel = (self.Kp_angular * error +
self.Ki_angular * self.integral_error +
self.Kd_angular * derivative)
```

Corrección de rumbo durante avances

Durante los desplazamientos lineales, el robot puede desviarse de la trayectoria recta debido a diferencias de fricción entre las ruedas o a imprecisiones en la simulación. Para corregir esto, se implementa un **controlador proporcional de rumbo** que actúa sobre la velocidad angular mientras el robot avanza:

$$\omega_{correccion} = K_{heading} \cdot \left(\theta_{objetivo} - \theta_{actual}\right) \tag{1.5}$$

con $K_{heading} = 1{,}5$. El ángulo objetivo $\theta_{objetivo}$ es la orientación que tenía el robot al comenzar el tramo recto. Si el robot se desvía a la derecha, el error es positivo y el controlador gira ligeramente a la izquierda, y viceversa. Esto garantiza que el robot mantenga una trayectoria recta real, no solo una velocidad lineal constante.

Además, la velocidad lineal se calcula de forma proporcional a la distancia restante:

$$v = \text{mín}\left(v_{max},\ \text{máx}\left(0{,}05,\ d_{restante} \cdot 1{,}0\right)\right) \tag{1.6}$$

Esto hace que el robot arranque a velocidad máxima ($0{,}2\,m/s$) y vaya frenando progresivamente conforme se acerca al punto objetivo, evitando que se pase por inercia.

```
# Avance con correccion de rumbo
remaining = self.target_distance - traveled
linear_vel = min(self.max_linear, max(0.05, remaining * 1.0))

heading_error = self.normalize_angle(self.target_yaw - self.
    yaw)
angular_vel = self.Kp_heading * heading_error

twist.linear.x = linear_vel
twist.angular.z = angular_vel
```

Estructura del nodo

El nodo `movimiento_controlado.py` utiliza la misma arquitectura de máquina de estados basada en timer (20 Hz) que el nodo de aparcamiento. Al inicio, construye una lista de pasos según el movimiento solicitado. Por ejemplo, para el cuadrado la lista es:

`[avanzar 1m, girar 90°, avanzar 1m, girar 90°, avanzar 1m, girar 90°, avanzar 1m, girar 90°]`

Para el infinito (forma de pajarita), la secuencia replica exactamente la del Ejercicio 1: avanzar $0{,}5\,m$, girar -120, avanzar la diagonal de $1{,}0\,m$, girar $+120$, avanzar $0{,}5\,m$, girar $+120$, avanzar la diagonal de $1{,}0\,m$ y girar -120 para recuperar la orientación original. La diferencia es que cada avance y cada giro se ejecutan con el controlador PID en vez de con velocidad fija por tiempo.

En cada tick del timer, el `control_loop` evalúa si el paso actual ha finalizado (error dentro de la tolerancia de $0{,}02\,m$ para distancias y $0{,}02\,rad$ para ángulos). Si es así, avanza al siguiente paso; si no, aplica el controlador correspondiente. Al completar todos los pasos, el timer se cancela y el robot se detiene.

Parámetros del controlador

Parámetro	Valor	Descripción
K_p (angular)	2.0	Ganancia proporcional para giros
K_i (angular)	0.01	Ganancia integral para giros
K_d (angular)	0.1	Ganancia derivativa para giros
$K_{heading}$	1.5	Corrección de rumbo en avance
v_{max}	0.2 m/s	Velocidad lineal máxima
ω_{max}	0.5 rad/s	Velocidad angular máxima
Tolerancia distancia	0.02 m	Error aceptable en avances
Tolerancia ángulo	0.02 rad	Error aceptable en giros ($\approx$ 1,1)

Cuadro 1.1: Parámetros del controlador PID implementado en `movimiento_controlado.py`

Resultados y comparativa

Al repetir los movimientos del Ejercicio 1 con el controlador PID, se observa una mejora significativa en la precisión de las trayectorias. El robot cierra las figuras (cuadrado, triángulo, infinito) regresando muy cerca del punto de origen, a diferencia del control en lazo abierto donde el error acumulado era evidente.

Para ejecutar el nodo:

```
# Movimiento lineal (2m)
ros2 run p3_pkg movimiento_controlado --ros-args -p movimiento
    :=0
# Triangulo
ros2 run p3_pkg movimiento_controlado --ros-args -p movimiento
    :=1
# Cuadrado
ros2 run p3_pkg movimiento_controlado --ros-args -p movimiento
    :=2
# Infinito
ros2 run p3_pkg movimiento_controlado --ros-args -p movimiento
    :=3
```

2. Parte 2: Resolución de un Laberinto

En esta parte se utiliza el Turtlebot 3 equipado con un sensor LiDAR para resolver laberintos de manera autónoma. Se emplea el paquete `maze_pkg`, que contiene dos nodos: `res_maze.py` (algoritmo *left-wall following*) y `pledge_maze.py` (Algoritmo de Pledge).

2.1 Preguntas teóricas previas

Pregunta 12: Topic y tipo de mensaje del LiDAR

El topic asociado al LiDAR del Turtlebot 3 es `/scan`. Lo verificamos con:

```
ros2 topic list
ros2 topic type /scan
```

El tipo de mensaje es `sensor_msgs/msg/LaserScan`, con los campos principales:

- `angle_min`, `angle_max`, `angle_increment`: rango y resolución angular.
- `range_min`, `range_max`: límites de distancia medibles.
- `ranges[]`: array con las distancias medidas en cada ángulo.
- `intensities[]`: intensidades de reflexión (vacío en simulación).

Pregunta 13: Rangos de distancia, ángulo de escaneo y origen de referencia

Los valores del LiDAR del Turtlebot 3 Waffle (sensor LDS-01) son:

Parámetro	**Waffle**	**Burger**
Rango mínimo	0.12 m	0.12 m
Rango máximo	3.5 m	3.5 m
Ángulo de escaneo	360°	360°
Resolución angular	1° (360 rayos)	1° (360 rayos)

Cuadro 2.1: Características del sensor LiDAR LDS-01 del Turtlebot 3

```
nickhernd@docker-desktop:/workspace$ ros2 topic echo /scan --
    once
header:
stamp:
sec: 13
nanosec: 113000000
frame_id: base_scan
angle_min: 0.0
angle_max: 6.28000020980835
angle_increment: 0.01749303564429283
time_increment: 0.0
scan_time: 0.0
range_min: 0.11999999731779099
range_max: 3.5
ranges:
- 2.4729459285736084
```

El origen de referencia es el propio sensor, montado en la parte superior del robot. El índice 0 del array `ranges[]` corresponde a la dirección de avance (*frente*), y los índices crecen en sentido antihorario: índice 90 $\approx$ izquierda, 180 $\approx$ atrás, 270 $\approx$ derecha.

Para el algoritmo se utilizan los siguientes sectores angulares:

- **Sector frontal**: índices correspondientes a $[-20, +20]$.
- **Sector izquierdo**: índices correspondientes a $[75, 105]$.

2.2 Ejercicio 1: Resolución del Laberinto 1 — Algoritmo Wall Follower

Se ha desarrollado el nodo `res_maze.py` dentro del paquete `maze_pkg`. El algoritmo implementado es *left-wall following* con una máquina de tres estados:

- **FIND_WALL**: el robot avanza en línea recta hasta detectar una pared al frente ($d_{\text{front}} < 0{,}45\,\text{m}$).
- **TURN_RIGHT**: el robot gira a la derecha sobre sí mismo hasta que el frente queda libre. Existe un contador mínimo `MIN_TURN_TICKS = 15` para evitar oscilaciones.
- **FORWARD**: avanza siguiendo la pared izquierda mediante un controlador proporcional. El error se define como $e = d_{\text{left}} - d_{\text{desired}}$; si la pared está lejos el robot gira a la izquierda y si está cerca gira a la derecha.

La función principal del nodo es el *callback* del LiDAR:

```python
def scan_callback(self, msg: LaserScan):
    front = min(safe(i) for i in range(-20, 21))
    left  = min(safe(i) for i in range(75, 106))

    if self.state == 'FIND_WALL':
```

```
        if front < FRONT_THRESH:
            self.state = 'TURN_RIGHT'
        else:
            cmd.linear.x = LINEAR_SPEED

    elif self.state == 'TURN_RIGHT':
        cmd.angular.z = -ANGULAR_SPEED
        self.turn_ticks += 1
        if self.turn_ticks >= MIN_TURN_TICKS and front >
           FRONT_THRESH + 0.15:
            self.state = 'FORWARD'

    else:  # FORWARD
        if front < FRONT_THRESH:
            self.state = 'TURN_RIGHT'
        else:
            error = left - WALL_DIST
            correction = max(-ANGULAR_SPEED, min(ANGULAR_SPEED,
               KP * error))
            cmd.linear.x  = LINEAR_SPEED
            cmd.angular.z = correction
```

Los parámetros principales del nodo son:

Parámetro	**Valor**	**Descripción**
`LINEAR_SPEED`	0.12 m/s	Velocidad lineal de avance
`ANGULAR_SPEED`	0.40 rad/s	Velocidad angular de giro
`WALL_DIST`	0.35 m	Distancia deseada a la pared izquierda
`FRONT_THRESH`	0.45 m	Umbral de detección de obstáculo al frente
`KP`	0.8	Ganancia del controlador proporcional
`MIN_TURN_TICKS`	15	Iteraciones mínimas en estado de giro

Cuadro 2.2: Parámetros del nodo `res_maze.py`

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_1.launch.py

# Terminal 2 - Solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg res_maze
```

El algoritmo Wall Follower resuelve correctamente el laberinto 1, ya que todas las paredes interiores están conectadas al perímetro exterior, condición suficiente para que el seguimiento de pared sea completo.

2.3 Ejercicio 2: Laberinto 2 y análisis de limitaciones del Wall Follower

Se ejecuta el mismo nodo `res_maze.py` con el segundo mapa, que presenta una topología diferente:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_2.launch.py
ros2 run maze_pkg res_maze
```

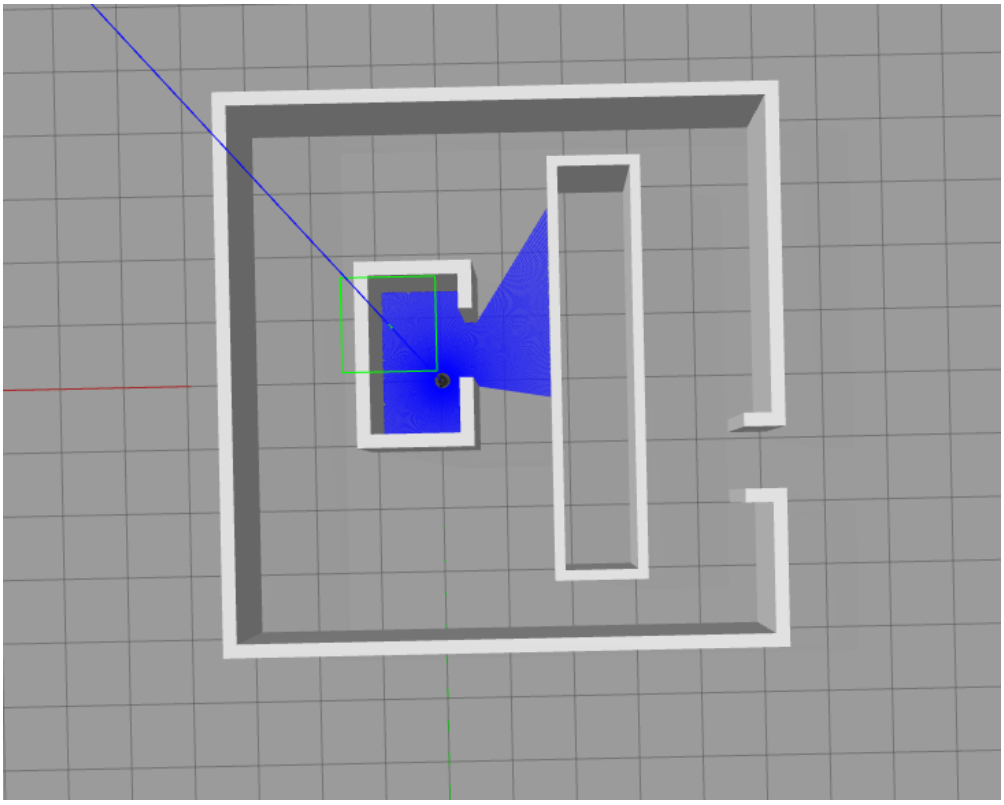

Figura 2.1: Turtlebot 3 Waffle atrapado en el laberinto 2: el robot orbita indefinidamente alrededor de la isla interior

Pregunta 14: Problemática observada en el laberinto 2

El laberinto 2 contiene **paredes interiores no conectadas al perímetro exterior** (islas). El algoritmo Wall Follower falla en este escenario porque:

1. El robot detecta la isla, la toma como pared de referencia y comienza a seguirla por la izquierda.
2. Al ser la isla un obstáculo cerrado (sin conexión a la salida), el robot completa el perímetro de la isla y vuelve al punto de inicio del seguimiento.
3. El proceso se repite indefinidamente: el robot queda atrapado en un bucle infinito rodeando la isla, sin llegar nunca a la salida.

Esta es la limitación fundamental del algoritmo: **solo garantiza la solución si el muro seguido está conectado topológicamente con la salida**.

Pregunta 15: ¿Es el robot capaz de resolver este laberinto? ¿Qué información necesita?

Con el algoritmo Wall Follower tal como está implementado, **el robot no es capaz de resolver el laberinto 2**. El robot queda atrapado orbitando alrededor de la isla interior sin encontrar la salida.

Para resolver este tipo de laberintos es necesario el **Algoritmo de Pledge** [1], una variante del Wall Follower que incorpora un *contador de giros netos*. La idea clave es: si el contador de giros regresa a cero mientras el camino está despejado, el robot abandona el muro y retoma la dirección original. Esto permite al robot *escapar* de las islas en lugar de orbitar alrededor de ellas. La implementación de este algoritmo se describe en el siguiente ejercicio.

2.4 Ejercicio 3: Algoritmo de Pledge para el Laberinto 2

Para resolver el laberinto 2 se ha implementado el nodo `pledge_maze.py` dentro del paquete `maze_pkg`. Este nodo implementa el **Algoritmo de Pledge**, descrito en el pseudocódigo siguiente:

Algoritmo de Pledge (seguimiento pared izquierda)

1. `contador` $\leftarrow 0$
2. **repetir:**
3. Avanzar en línea recta (*estado* NORTE) hasta encontrar un muro
4. Comenzar a seguir el muro izquierdo (*estado* MURO)
5. Incrementar/decrementar `contador` con cada giro
6. Si `contador` $= 0$ y el frente está libre $\Rightarrow$ volver al paso 3
7. **hasta:** salida encontrada

Cuadro 2.3: Pseudocódigo del Algoritmo de Pledge

Implementación en ROS 2

La clase `PledgeFinal` suscribe a `/scan` (LiDAR) y a `/odom` (odometría) para extraer el ángulo *yaw* actual mediante:

```
def odom_cb(self, msg):
    q = msg.pose.pose.orientation
    self.curr_yaw = math.atan2(
        2.0*(q.w*q.z + q.x*q.y),
        1.0 - 2.0*(q.y*q.y + q.z*q.z))

    if self.init_yaw is None:
```

```
        self.init_yaw = self.curr_yaw
        return

    # Contador de cuadrantes (-2, -1, 0, 1, 2)
    diff = self.normalize_angle(self.curr_yaw - self.init_yaw)
    current_q = int(round(diff / (math.pi / 2)))

    if current_q != self.last_q:
        step = current_q - self.last_q
        if step > 2:  step -= 4
        if step < -2: step += 4
        self.contador += step
        self.last_q = current_q
```

La máquina de estados principal, en el *callback* del LiDAR, tiene dos estados: `NORTE` (avanzar recto manteniendo la orientación inicial) y `MURO` (seguir la pared izquierda):

```
def scan_cb(self, msg):
    dist_frontal = self.get_dist_at_angle(msg, 0)
    dist_izq     = self.get_dist_at_angle(msg, 90)

    if self.state == 'NORTE':
        if dist_frontal < 0.45:
            self.state = 'MURO'  # Encontro muro -> empezar a seguirlo
        else:
            # Controlador P de rumbo: mantener orientacion inicial
            err = self.normalize_angle(self.init_yaw - self.curr_yaw)
            cmd.linear.x  = 0.15
            cmd.angular.z = 0.8 * err

    elif self.state == 'MURO':
        # Condicion de escape Pledge: contador=0 y orientacion inicial recuperada
        dist_norte = abs(self.normalize_angle(self.curr_yaw - self.init_yaw))
        if self.contador == 0 and dist_norte < 0.15 and dist_frontal > 0.6:
            self.state = 'NORTE'  # Escapar del muro
            return

        # Navegacion: seguir pared izquierda
        if dist_frontal < 0.35:
            cmd.angular.z = -0.5            # Girar derecha (resta al contador)
        elif dist_izq > 0.6:
            cmd.linear.x  = 0.1
            cmd.angular.z = 0.4             # Girar izquierda (suma al contador)
        else:
```

```
            cmd.linear.x  = 0.12
            cmd.angular.z = 1.3 * (dist_izq - 0.35)  # Control
                proporcional
```

La condición de escape es clave: cuando el `contador` vuelve a 0 **y** el robot ha recuperado su orientación original **y** el frente está libre, el robot abandona el muro. Esto garantiza que no se trata de un punto de paso al azar, sino de un estado en que el robot ha completado un número neto de giros igual a cero — condición necesaria para haber escapado de la isla.

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_2.launch.py

# Terminal 2 - Pledge solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg pledge_maze
```

El algoritmo de Pledge resuelve correctamente el laberinto 2. Cuando el robot rodea la isla por primera vez, el contador acumula giros; al completar el perímetro el contador vuelve a cero y el robot escapa en línea recta hacia la salida, en lugar de repetir el bucle.

El enlace al vídeo de la ejecución se puede encontrar en: `ENLACE_VIDEO_MAZE2_PLEDGE`

2.5 Ejercicio 4: Comparativa Waffle vs. Burger en el Laberinto 1

Se repite el Ejercicio 1 usando el modelo `Burger`:

```
export TURTLEBOT3_MODEL=burger
ros2 launch maze_pkg maze_1.launch.py
ros2 run maze_pkg res_maze
```

Pregunta 16: Diferencias observadas entre Waffle y Burger

Las diferencias observadas entre ambos modelos al resolver el laberinto son:

- **Dimensiones físicas**: el Burger (138 mm de diámetro) es más pequeño que el Waffle (281 mm), lo que le proporciona mayor holgura en pasillos estrechos y reduce el riesgo de colisiones laterales.
- **LiDAR**: ambos modelos utilizan el mismo sensor LDS-01 con cobertura de 360° y resolución de 1°, por lo que la percepción es equivalente y el mismo código funciona sin modificaciones.

- **Dinámica de giro**: el Burger tiene menor separación entre ruedas (*wheel separation*), lo que permite giros más cerrados pero también lo hace más sensible a acumulación de error odométrico.
- **Estabilidad**: el Waffle, más pesado (1.8 kg vs. 0.9 kg), es más estable a las velocidades empleadas. El Burger tiende a oscilar ligeramente más con los mismos parámetros del controlador proporcional.
- **Comportamiento en el laberinto**: ambos robots resuelven el laberinto 1 correctamente con los mismos parámetros, aunque el Burger lo hace con mayor margen de distancia a las paredes al ser más estrecho.

2.6 Ejercicio 5: Laberinto propio (maze_3)

Diseño del laberinto

Se ha diseñado un laberinto personalizado de **24×24 metros** (576 celdas de 1×1 m). Las características del diseño son:

- **Tamaño**: 24×24 m, 576 celdas, pasillos de 1 m de ancho y paredes de 0.15 m de grosor.
- **Una única salida**: hueco de 2 m en la pared norte, en $x \in [11, 13]$. No hay entrada; el robot se spawna directamente dentro en la posición (1,5, 1,5).
- **Sin islas simples**: la estructura base garantiza que todas las celdas son accesibles y las paredes forman un árbol generador.
- **Bucles añadidos**: se eliminaron manualmente 32 segmentos de pared adicionales para crear bucles internos, lo que hace que el Wall Follower simple quede atrapado y requiera el Algoritmo de Pledge para escapar.
- **Complejidad**: 288 segmentos de pared en total.

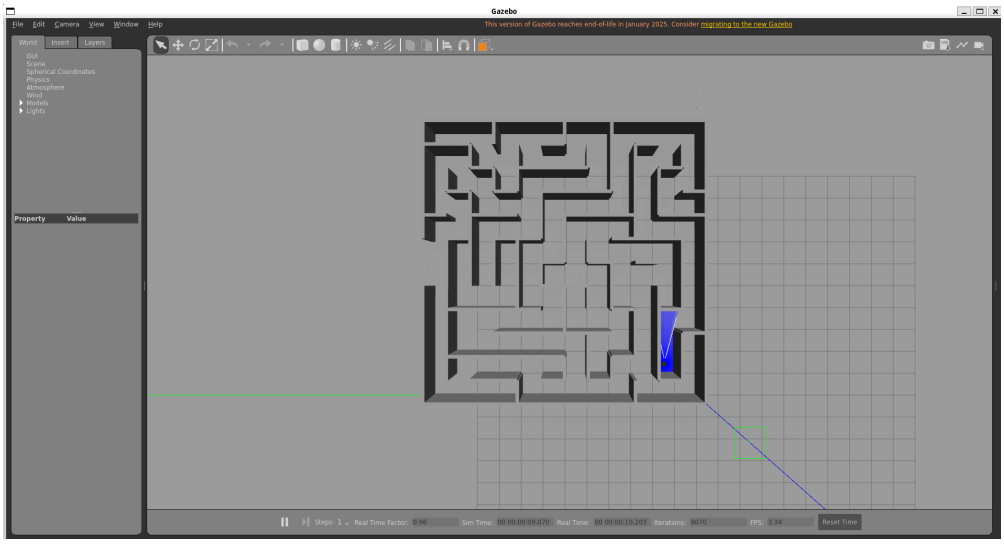

Figura 2.2: Vista cenital del laberinto propio (maze_3, 24×24 m) en Gazebo

Resolución con el Algoritmo de Pledge

Para lanzar el laberinto personalizado con el Algoritmo de Pledge:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_3.launch.py

# Terminal 2 - Pledge solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg pledge_maze
```

El Algoritmo de Pledge resuelve satisfactoriamente el laberinto 3. Los bucles introducidos en el diseño atrapan al Wall Follower simple, pero el contador de cuadrantes del Pledge permite al robot escapar de cada bucle en cuanto los giros netos vuelven a cero y la dirección norte está libre. Dado el tamaño del laberinto (24 m), el robot recorre un camino significativamente más largo antes de encontrar la salida, lo que pone de manifiesto la robustez del algoritmo frente a laberintos de gran escala.

Bibliografía

[1] R. Klein and T. Kamphans, *Pledge's Algorithm — How to Escape from a Dark Maze*, pp. 69–75, Springer Berlin Heidelberg, 2011. https://doi.org/10.1007/978-3-642-15328-0_8